Learning to Race with PPO

How Policy Network Size Affects Performance, Convergence, and Training Time

Yagiz Tansu

Reinforcement Learning — AA 2025–26

Contents

1 Abstract

A reinforcement learning agent is trained to drive a car around a 2D racetrack in minimum time. The agent uses **Proximal Policy Optimization (PPO)** with a continuous Gaussian policy. The car is controlled through throttle/brake and steering.

The main question of this project is simple: **does a bigger neural network help?** Three policy network sizes are tested and three quantities are measured: (1) final lap performance, (2) how many episodes are needed to learn, and (3) real wall-clock training time.

The environment was custom-built with Gymnasium. It uses a kinematic bicycle model for physics and Frenet coordinates for the state. The agent sees its lateral offset, heading error, speed, and upcoming track curvature — but not its absolute position on the map.

The results show that all three network sizes learn to complete a full lap on a **fixed reference circuit** with similar final return (about 1139). Lap times differ by less than 0.1 simulated seconds. The smallest network `[16]` is selected as the winning architecture on the reference benchmark (mean completion, return, and lap time over four seeds). It also converges in the **fewest episodes** (261 $\pm$ 34) and the **shortest wall-clock time** (149 $\pm$ 16 s). When the winning checkpoint (`arch16_seed3_random`) is tested on **held-out procedural tracks** never seen in training (seeds 1000–1009), it completes **100%** of evaluation laps — showing that random-track training generalises to new road layouts.

2 Introduction

2.1 Motivation

Autonomous vehicles must choose throttle, brake, and steering many times per second. Good control is not a single decision — it depends on what happened earlier in the lap. For example, too much speed before a corner can push the car off the track later.

Reinforcement learning (RL) is well suited to this kind of problem. An agent interacts with an environment, receives rewards, and learns a policy over many trials. No full driving model needs to be programmed by hand.

This project applies RL to a **simplified 2D car racing task**. One car must drive around a **closed track** as fast as possible while staying on the road. The setup is simpler than a real car simulator, but it still requires sequential decision-making, continuous control, and a clear performance goal.

2.2 Goal

The training objective is to **complete one full lap in minimum time**.

A reference racing line is **not** hand-coded. The agent must learn when to accelerate, brake, and steer from the reward signal alone. The policy is trained with **Proximal Policy Optimization (PPO)** and a neural network that maps observations to actions.

The full pipeline was implemented in Python: track generation, vehicle dynamics, a custom **Gymnasium** environment, training with **Stable-Baselines3**, evaluation, and result plots.

2.3 Research questions

The RL algorithm and hyperparameters are kept **identical** across all runs. The only variable is **policy network size** — the number of neurons in the hidden layers. Three sizes are compared: `[16]`, `[64, 64]`, and `[256, 256]`.

The project addresses three questions:

1. **Final performance** — After training, how well does the agent drive? This is measured by evaluation return, lap completion rate, and lap time on a completed lap.
2. **Episodes to convergence** — How many training episodes are needed before performance stabilises? Convergence is defined as the first episode where a 50-episode rolling-mean return reaches 90% of the run's final rolling mean.
3. **Wall-clock time** — How long does training take in real seconds on the training machine? This matters for practical cost, not only learning speed in episodes.

Each network size is trained with **four random seeds** (0, 1, 2, 3). Results are reported as **mean ± standard deviation** across seeds.

Training uses **random procedural tracks**: each episode samples a new circuit from a pool of 50 layouts (seeds 0–49). **Evaluation** uses two separate test sets:

1. **Reference circuit** — a fixed hand-designed track (`track()`) used to compare architectures fairly (same road for every run).
2. **Held-out circuits** — ten procedural tracks (seeds 1000–1009) never used in training, used only to test generalisation of the winning checkpoint.

3 Background

3.1 Reinforcement learning basics

An RL agent interacts with an **environment** over many **steps**. At each step:

1. The agent observes state s_t.
2. It chooses action a_t.
3. The environment returns reward r_t and the next state s_{t+1}.

The agent's goal is to maximize the **total return** (sum of rewards) over an **episode** — one run from start to finish.

3.2 Policy gradient methods

A **policy** is a function $\pi(a \mid s)$ that maps states to actions. In **policy gradient** methods the policy is represented with a neural network and its weights are updated to increase expected return.

This project uses **continuous actions** (throttle and steering), so the policy outputs a **Gaussian distribution** over actions. During training, actions are **sampled** from that distribution to explore. During evaluation, the **mean** action is used only.

3.3 PPO (Proximal Policy Optimization)

PPO is a popular on-policy algorithm. It updates the policy many times on recent data, but limits how much the policy can change in one update. This makes training more stable than vanilla policy gradients.

PPO from **Stable-Baselines3** is used with an MLP policy (`MlpPolicy`). The same network architecture is used for both the **actor** (actions) and the **critic** (value estimate).

3.4 Why this problem fits RL

- The task is sequential: early mistakes (e.g. too much speed before a corner) cause later failure.
- The reward depends on progress along the track, not on a single correct answer.
- The agent must balance speed and safety from experience.

4 Environment Design

4.1 Track generation

The track is a **closed loop**. First, 8–12 control points are placed in the plane. Then a **periodic cubic spline** is fitted through them, and the centerline is resampled at uniform **arc-length** spacing. Each resampled point stores:

- cumulative arc length s
- unit tangent vector
- signed curvature κ (positive = left turn, negative = right turn)

The road has constant half-width (6 m). A car is on-track when its lateral offset $|d| \leq$ half-width.

Three track modes are used in the project:

Mode	How it is built	Role
Reference circuit	Hand-placed control points (~1046 m)	Fair benchmark for architecture comparison
Training pool	Procedural tracks from seeds 0–49 (700–1400 m)	One random layout per training episode
Held-out test set	Procedural tracks from seeds 1000–1009	Generalisation test only

Figure 1 shows the ten held-out test tracks. They are longer and differently shaped from the reference circuit; the agent never sees them during training.

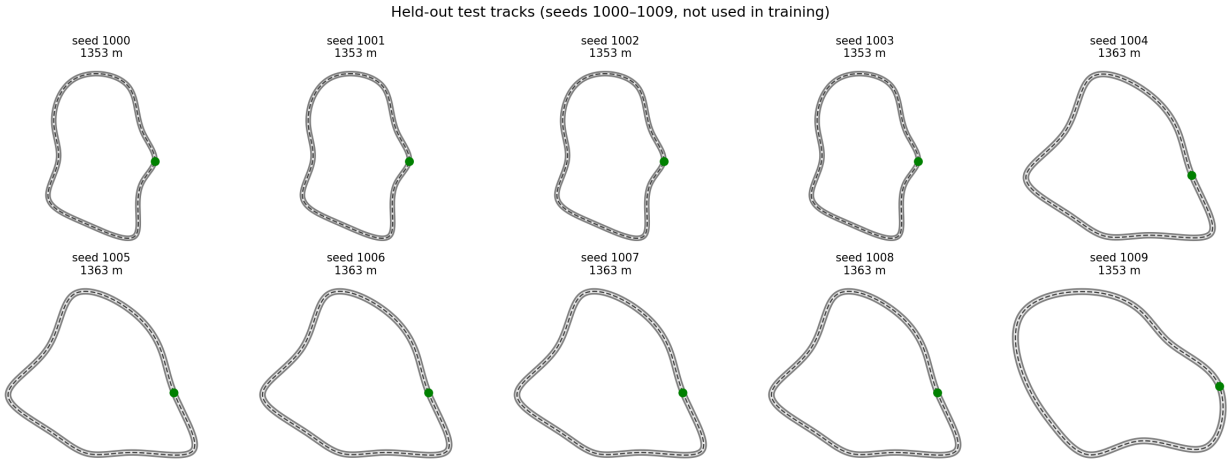

Figure 1: Held-out test tracks (seeds 1000–1009, not used in training).

How to read this figure: Each panel is a top-down view of a track centerline. Length varies because procedural generation produces different circuit sizes. The agent always sees the road relative to itself (Frenet frame), not absolute (x, y) position.

4.2 Car dynamics

A **kinematic bicycle model** is used with forward Euler integration and $dt = 0.05$ s (20 Hz). Main constants:

Parameter	Value
Max speed $v_{\max}$	30 m/s
Max acceleration	8 m/s^2
Max steering	0.45 rad
Wheelbase L	3.0 m

Given acceleration a and steering δ, the model updates position, heading, and speed. Speed is clipped to $[0, v_{\max}]$.

4.3 Frenet coordinates

Cartesian position (x, y) is hard for the agent to interpret. Positions are therefore converted to **Frenet** track-relative coordinates:

Symbol	Meaning
s	Arc length along centerline from the start
d	Lateral offset from centerline (+ left, - right)
θ_e	Heading error vs. track tangent

Lookahead curvatures are also computed at distances $[5, 10, \ldots, 110]$ m ahead so the agent can anticipate corners.

Design choice: s is **not** included in the observation. Absolute progress does not generalize well across different tracks. Offset, heading error, speed, and upcoming curvature do.

4.4 Observation space (14 numbers)

All features are normalized to roughly $[-1, 1]$:

Index	Feature
0	d/half_width
1–2	$\sin(\theta_e)$, $\cos(\theta_e)$
3	$v/v_{\max}$
4–13	Signed curvature at 10 lookahead distances $\times$ scale factor

4.5 Action space (2 numbers)

Both actions lie in $[-1, 1]$:

- `action[0]` $\rightarrow$ acceleration $= 8.0 \times$ action[0] m/s^2
- `action[1]` $\rightarrow$ steering $= 0.45 \times$ action[1] rad

4.6 Reward function

Each step:

$$\text{reward} = 1.0 \times \Delta s - 0.01$$

Event	Effect
Progress Δs	Reward for moving forward along the track
Time penalty -0.01	Small cost per step to discourage stopping
Off-track ($\|d\| >$ half-width)	-80 penalty, episode **terminated**
Full lap completed	$+100$ bonus, episode **terminated**
2000 steps reached	Episode **truncated** (not terminated)

Gymnasium separates **terminated** (real end: crash or lap done) from **truncated** (time limit). PPO uses this distinction when bootstrapping value targets.

5 Training and Evaluation

5.1 Network architectures

Three hidden-layer sizes are tested (same hyperparameters otherwise):

Label	Hidden layers	Role in study
Tiny	`[16]`	Very low capacity
Small	`[64, 64]`	Medium
Large	`[256, 256]`	High

5.2 Training procedure

- Algorithm: PPO (`stable-baselines3`), default hyperparameters unless noted in `config.json`
- Parallel envs: 8 subprocess environments (`SubprocVecEnv`)
- Total training steps: 1,000,000 per run (configurable via CLI)
- Seeds: 0, 1, 2, 3 per architecture
- Track mode: **random** — each `reset()` samples a new circuit from training pool seeds 0–49

Each episode is logged to `progress.csv`: return, length, mean log std of the Gaussian policy, wall-clock time.

5.3 Evaluation procedure

Evaluation is split into two stages:

Stage 1 — Architecture comparison (all 12 runs). Each saved model is evaluated on the **reference circuit** with `deterministic=True` (Gaussian mean action, no exploration noise). Metrics: mean return, completion rate, lap time. Architectures are ranked by mean completion, then return, then lap time across four seeds.

Stage 2 — Generalisation (winning checkpoint only). The best seed of the winning architecture is evaluated on the **held-out tracks** (seeds 1000–1009). This checks whether random-track training transfers to unseen road layouts.

During training, actions are **stochastic** (sampled). For evaluation, `deterministic=True` is set and the Gaussian **mean** μ is always used.

6 Experiments and Results

A grid of **3 architectures × 4 seeds = 12 training runs** was executed with **random-track training** (one new circuit per episode). After training, `python -m src.plots` evaluates all models on the **reference circuit**, builds architecture-comparison figures, then evaluates the **winning checkpoint** on held-out tracks. All figures are written to `experiments/figures/`.

Convergence definition (used in Figures 3–4): first episode where the 50-episode rolling-mean return reaches **90%** of that run's own final 50-episode rolling mean.

6.1 Learning curves

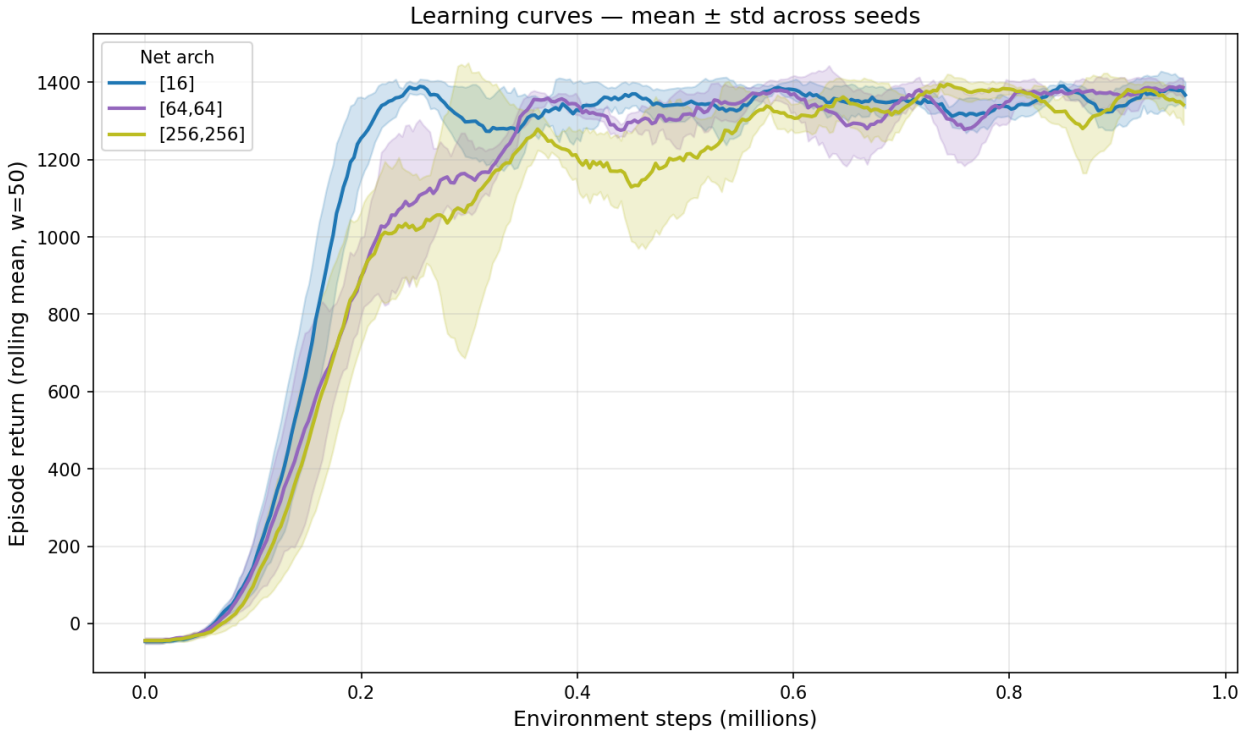

Figure 2: Rolling-mean episode return vs environment steps (mean ± std across seeds).

How to read: The x-axis is cumulative environment steps. The y-axis is episode return, smoothed with a 50-episode rolling mean. Each colour is one network size; shaded bands show variability across the four seeds.

Results: All three architectures show a clear upward trend and eventually reach returns near 1100–1140 on completed laps. Because each training episode uses a **different random track**, curves are noisier than in fixed-track training, but all sizes reach similar final performance. Extra capacity does not produce a much higher return.

6.2 Final performance (reference circuit)

How to read: Bars show **deterministic** evaluation on the **reference circuit** after random-track training: mean return and lap completion rate, averaged over four seeds (error bars = std across seeds).

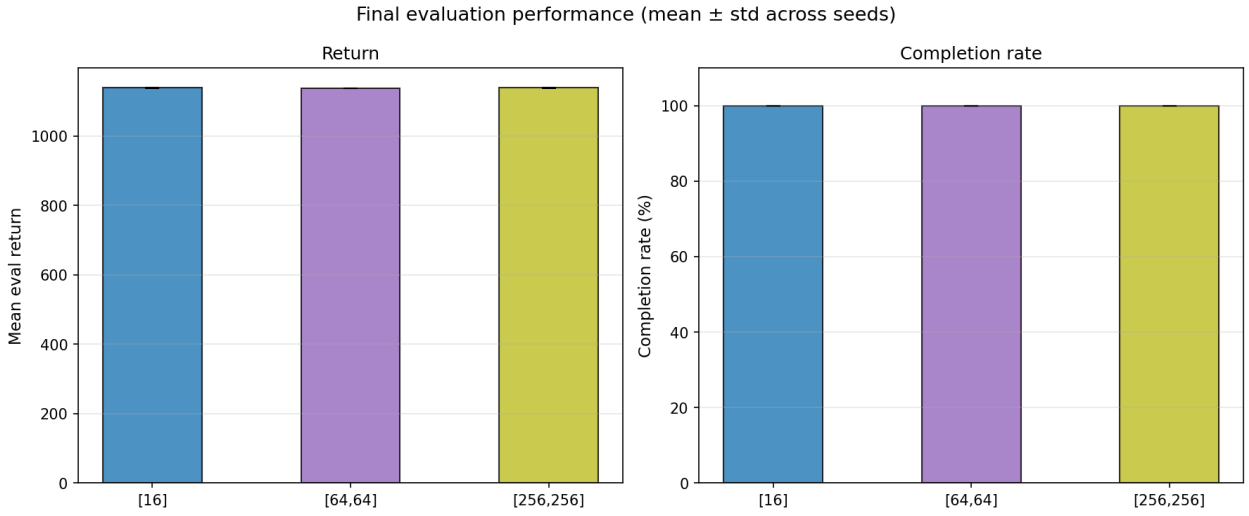

Figure 3: Evaluation return and lap completion rate per network size on the reference circuit.

Results: Every architecture reaches **100% completion** on all seeds. Mean return is about **1139** for all three (differences under 1 point). Lap time is nearly identical (~**36.0 s** simulated for `[16]` and `[256, 256]`, ~**35.9 s** for `[64, 64]`). On this benchmark, **network size barely affects final driving quality**.

6.3 Convergence in episodes

How to read: Lower bars mean the rolling-mean return reached 90% of its final value in fewer training episodes.

Results: `[16]` converges in the **fewest episodes** (**261** ± **34**), followed by `[256, 256]` (**327** ± **43**) and `[64, 64]` (**348** ± **45**). With random-track training, the **smallest network is the most sample-efficient**. The medium network `[64, 64]` needs the most episodes on average — the opposite of what was observed under fixed-track training.

6.4 Convergence in wall-clock time

How to read: Same convergence rule as above, but measured in **real seconds** on the training machine.

Results: `[16]` is fastest (**149** ± **16 s**), then `[256, 256]` (**208** ± **30 s**), then `[64, 64]` (**228** ± **40 s**). All architectures have similar per-step cost (~**0.74–0.77 s** per 1k steps). The tiny network wins on both episode count and wall-clock time.

6.5 Policy exploration (sigma decay)

How to read: `mean_log_std` measures how much randomness remains in the Gaussian policy. It typically **decreases** as the agent becomes more confident.

Results: All architectures show exploration decaying over training. Random-track episodes produce noisier learning curves, but policy uncertainty still decreases steadily. By the end of training, all policies operate with low action noise in evaluation mode.

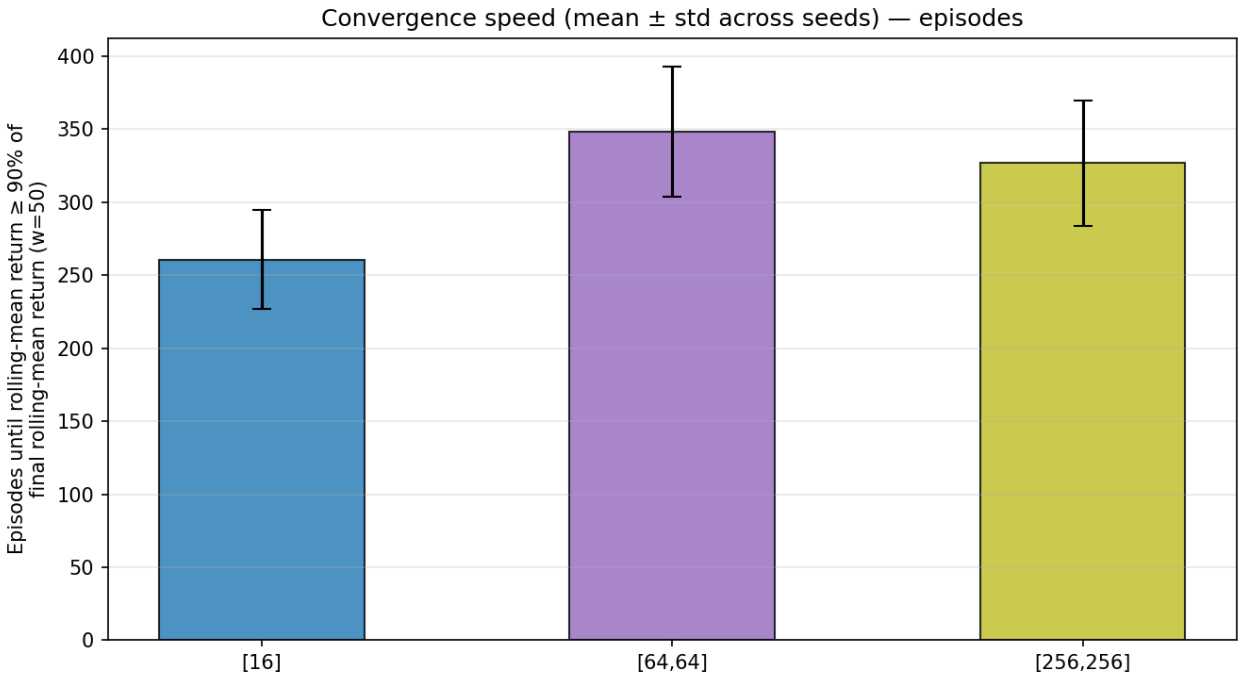

Figure 4: Episodes until convergence criterion is met (mean ± std).

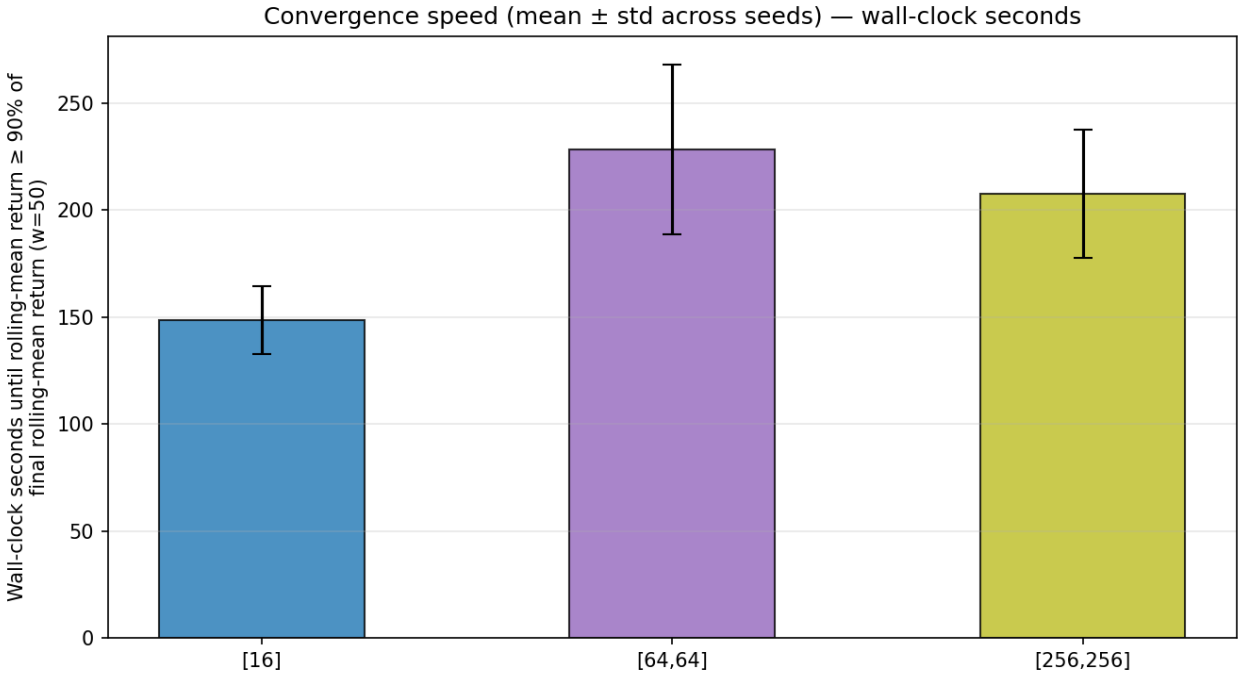

Figure 5: Wall-clock seconds until convergence (mean ± std).

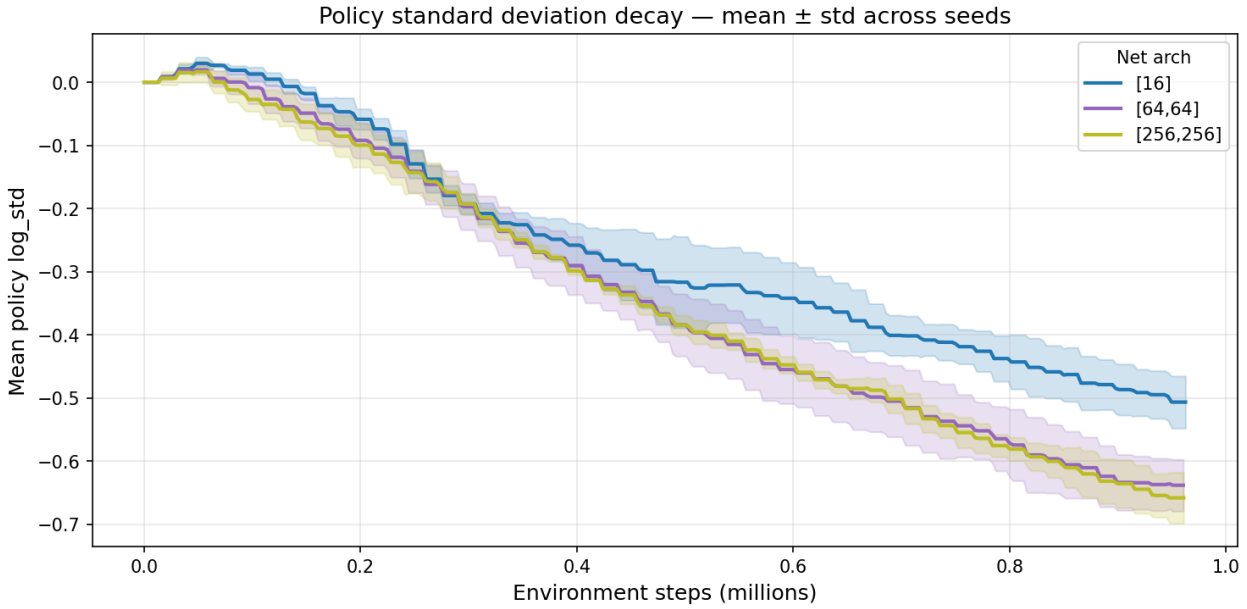

Figure 6: Policy log-standard-deviation vs training steps.

6.6 Summary table (reference circuit)

The table below is copied from `experiments/figures/results_table.md` (12 runs: 3 architectures × 4 seeds, evaluated on the reference circuit).

Convergence = first episode where 50-episode rolling-mean return ≥ 90% of the run's own final 50-episode rolling mean.

Net arch	Final return	Completion	Lap time (steps)	Lap time (sim s)	Episodes to converge	Wall-clock to converge (s)	Wall-clock / 1k steps (s)
[16]	1139.3 ± 0.5	100.0%	720	36.00 s	261 ± 34	149 ± 16	0.773 ± 0.010
[64,64]	1138.8 ± 0.0	100.0%	719	35.94 s	348 ± 45	228 ± 40	0.773 ± 0.022
[256,256]	1139.3 ± 0.5	100.0%	720	36.01 s	327 ± 43	208 ± 30	0.738 ± 0.023

Winning architecture (reference circuit, mean over seeds): `[16]`.

How to read the table: All rows complete the lap. Return and lap time are nearly identical across sizes. For convergence, `[16]` ranks first on both episodes and wall-clock time. Architecture ranking on the reference circuit is used only for the main PG-4 comparison; generalisation is tested separately below.

6.7 Generalisation to held-out tracks

After selecting `[16]` as the winning architecture, checkpoint **`arch16_seed3_random`** (best seed on the reference circuit) is evaluated on ten **held-out procedural tracks** (seeds 1000–1009).

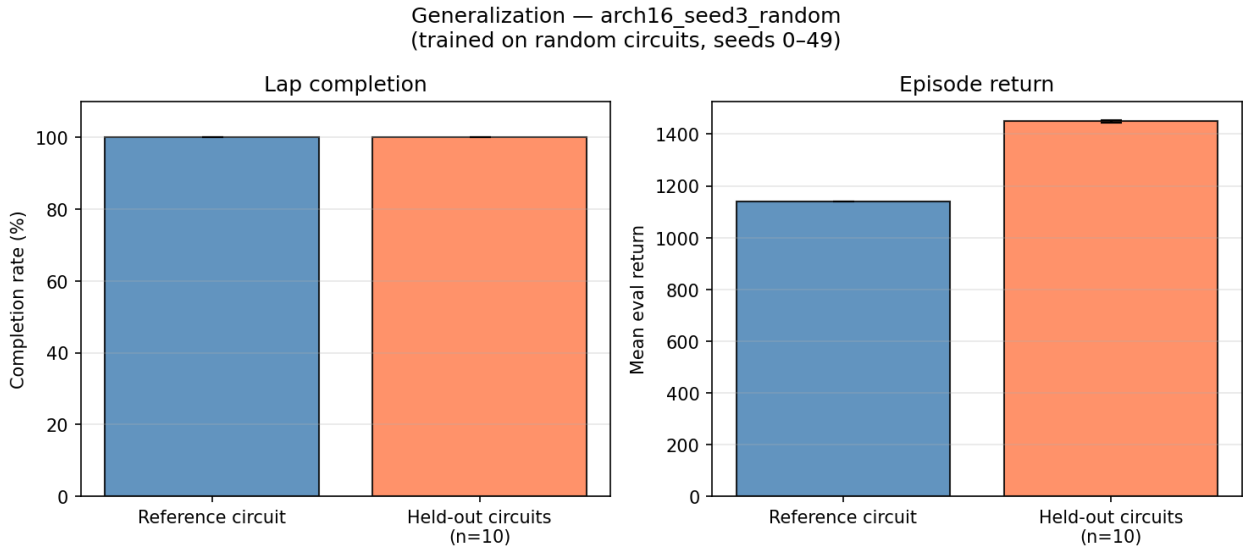

Figure 7: Reference circuit vs held-out completion and return for the winning checkpoint.

How to read: Left bar = reference circuit; right bar = mean over ten held-out tracks. Held-out tracks are longer (~1350–1360 m vs ~1046 m on the reference circuit), so returns are higher even when the agent completes every lap.

Results: The winning checkpoint achieves **100% completion** on the reference circuit and on **all ten held-out tracks**. Mean held-out completion is **100%**. Random-track training (pool 0–49) produces a policy that generalises to entirely new road layouts without retraining.

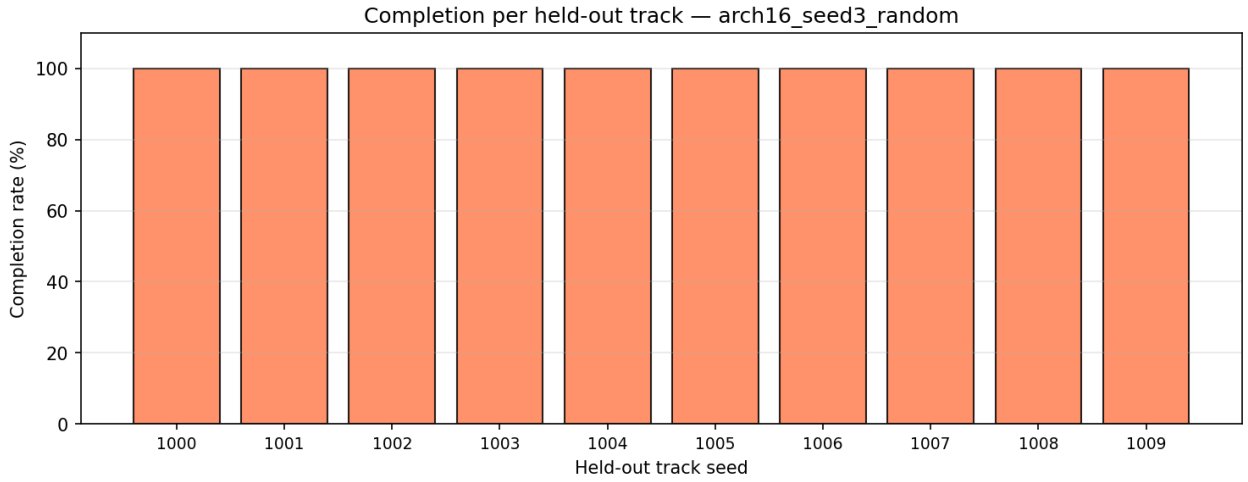

Figure 8: Completion rate per held-out track seed.

How to read: One bar per held-out track seed (1000–1009). All bars at 100% confirm consistent generalisation across different circuit lengths and shapes.

Held-out results (from `experiments/figures/generalization_table.md`):

Track	Seed	Length (m)	Completion	Mean return
Reference circuit	—	1046.0	100.0%	1139.8
Held-out (mean)	1000–1009	~1353–1363	100.0%	~1444–1455

7 Discussion

7.1 Network size vs performance

All three tested sizes — including the tiny `[16]` network — learn to complete the lap on the **reference circuit** with **100% success** and similar return (~1139). Final lap time differs by less than **0.1 simulated seconds** across architectures. **Extra parameters do not improve driving quality** once the network is large enough for this observation space.

7.2 Network size vs training cost

With **random-track training**, the cost ranking differs from fixed-track experiments:

- **Episodes:** `[16]` converges in the fewest episodes (261), making it the most sample-efficient.
- **Wall-clock:** `[16]` also reaches convergence in the shortest real time (149 s).
- **Medium network:** `[64, 64]` needs the most episodes (348) and the longest wall-clock time (228 s) — it does not offer a training-speed advantage here.

The smallest network is the best choice on both final benchmark performance and training efficiency for this setup.

7.3 Generalisation

Random-track training (pool 0–49) produces policies that transfer to **held-out circuits** (1000–1009). The winning checkpoint completes every evaluation lap on all ten unseen tracks. This confirms that Frenet-relative observations and curvature lookahead generalise across different track shapes and lengths, not just memorisation of one layout.

7.4 Limitations

- **Simplified physics** — kinematic bicycle, no tire slip dynamics.
- **Procedural track diversity** — training pool has 50 layouts; generalisation beyond similar procedural tracks is not tested.
- **Single machine** — wall-clock times depend on CPU/GPU and parallel env count.
- **Narrow performance range** — all successful policies achieve nearly the same return on the reference circuit, so differences are mainly in training cost, not final lap quality.

7.5 Reproducibility

```
python run_experiments.py              # 12 random-track runs
python -m src.plots                    # reference eval + generalisation figures
python -m src.train --net-arch 16 --seed 3 --track random --total-steps 1000000
python -m src.evaluate --run-name arch16_seed3_random --track-seed 1000
```

8 Conclusion

A custom Gymnasium racing environment was built with Frenet observations and PPO training. Agents were trained on **random procedural tracks** (one new circuit per episode) and evaluated on a **fixed reference circuit** for fair architecture comparison, plus **held-out tracks** for generalisation. The effect of policy network size was studied over 12 runs (3 architectures $\times$ 4 seeds, 1M steps each).

1. **Final performance (reference circuit)** — All three networks complete the lap on every seed with mean return ~1139. Lap times differ by less than 0.1 s. Network size barely affects final driving quality on the benchmark track. `[16]` is selected as the winning architecture (mean completion, return, and lap time over seeds).
2. **Episodes to convergence** — `[16]` converges in the fewest episodes (**261** $\pm$ **34**). `[64, 64]` needs the most (**348** $\pm$ **45**). With random-track training, the smallest network is the most sample-efficient.
3. **Wall-clock time** — `[16]` also reaches convergence fastest in real time (**149** $\pm$ **16 s**). For this task and training setup, a smaller network is the better practical choice.
4. **Generalisation** — The winning checkpoint (`arch16_seed3_random`) achieves **100% lap completion** on all ten held-out procedural tracks. Random-track training generalises to unseen road layouts.

Overall, for random-track racing the **tiny network** `[16]` offers the best balance: equal final performance on the reference circuit, fastest convergence, and strong generalisation to new tracks. Larger networks add training cost without improving results.